

Final Project: Assessment & Architecture Design

1. Phase 1: Assessment

1.1 Functional Domains

Four functional domains were identified from the app description: **Auth**, **Product**, **Cart**, and **Purchase**. Admin CRUD-on-products was folded into the Product domain rather than treated as separate because a domain is defined by the data it owns, not by who is allowed to touch it.

1.2 Data Ownership & Dependencies

Domain	Owns	Depends on
Auth	User accounts, credentials, roles (admin, user)	Nothing
Product	Catalog: name, description, price, stock, images	Auth (admin role required for create/update/delete; user role can view)
Cart	Cart items, quantities, owning user	Auth (identify user + role must be "user" specifically. admins are blocked from Cart) + Product (existence, stock, current price)
Purchase	Orders, transactions, payment records, receipts	Auth (identify user + confirm "user" role, same admin block as Cart) + Cart (items/qty being purchased) + Product (confirm final price/stock at purchase time)

1.3 Microservice Boundaries

All **4 domains** become **separate microservices**: **Auth**, **Product**, **Cart**, **Purchase**, a 1:1 mapping with no merging. **Cart vs. Purchase** was the borderline call, resolved in favor of full separation because:

- Cart is temporary, high-churn data; Purchase is permanent, immutable financial records that must stay isolated from Cart bugs
- Very different traffic/scaling profiles: Cart is hit constantly, Purchase is hit rarely but must prioritize reliability over speed

2. Phase 2: Architecture Design

2.1 Network Architecture: Subnets

Four subnets are used, per Oracle's official OKE network configuration guidance. Pods physically run on worker node hardware, but draw their IP addresses from the **separate Pods subnet** via a second VNIC attached to each node.

Subnet	Type	Purpose	CIDR	Range
API Endpoint	Public, regional	Kubernetes API access (kubectl)	10.0.0.0/28	10.0.0.0 – 10.0.0.15

Load Balancer	Public, regional	Public entry point for the app	10.0.0.16/28	10.0.0.16 – 10.0.0.31
Worker Nodes	Private, regional	Hosts the VMs running all pods	10.0.16.0/20	10.0.16.0 – 10.0.31.255
Pods	Private, regional	IP addresses for individual pods (VCN-native networking)	10.0.32.0/19	10.0.32.0 – 10.0.63.255

The Pods subnet is larger than the Worker Nodes subnet (/19 vs /20) per Oracle's requirement, since each node can host many pods, each needing its own IP address.

2.2 Gateways

Gateway	Attached to	Purpose
Internet Gateway	API Endpoint, Load Balancer subnets	Inbound/outbound internet access for public subnets
NAT Gateway	Worker Nodes, Pods subnets	Outbound-only internet access for private subnets
Service Gateway	Worker Nodes, Pods subnets	Private, fast path to OCI services (e.g. OCIR) without using NAT

2.3 Route Tables

Each subnet has its own route table.

Subnet	Destination	Target
API Endpoint (public)	0.0.0.0/0	Internet Gateway
Load Balancer (public)	0.0.0.0/0	Internet Gateway
Worker Nodes (private)	All <region> Services in Oracle Services Network	Service Gateway
Worker Nodes (private)	0.0.0.0/0	NAT Gateway
Pods (private)	All <region> Services in Oracle Services Network	Service Gateway
Pods (private)	0.0.0.0/0	NAT Gateway

2.4 Security Rules: Network Security Groups (NSGs)

Rules are enforced via NSGs rather than Security Lists. In this specific design the two would enforce identical traffic, since each subnet holds exactly one resource type. However, **NSGs** are used because they attach to individual resources rather than whole subnets, matching **Oracle's own recommendation** in every official rule table, decoupling security policy from network topology, and remaining **reusable** if the design changes later.

Kubernetes API Endpoint subnet — Ingress

Source	Port	Description
Worker Nodes CIDR	TCP 6443	Worker to API endpoint communication
Worker Nodes CIDR	TCP 12250	Worker to API endpoint communication
Pods CIDR	TCP 6443	Pod to API endpoint communication (VCN-native)

Pods CIDR	TCP 12250	Pod to API endpoint communication (VCN-native)
Worker Nodes CIDR	ICMP 3,4	Path MTU Discovery
Team IPs (optional)	TCP 6443	kubectrl / client access, not 0.0.0.0/0

Kubernetes API Endpoint subnet — Egress

Destination	Port	Description
OCI Services Network	TCP 443	API endpoint to OKE communication
Pods CIDR	ALL	API endpoint to pod communication (VCN-native)
Worker Nodes CIDR	ICMP 3,4	Path MTU Discovery
Worker Nodes CIDR	TCP 10250, ICMP	API endpoint to worker node communication

Worker Nodes subnet — Ingress

Source	Port	Description
Worker Nodes CIDR	ALL	Communication between worker nodes
Pods CIDR	ALL	Pods on one node reach pods on other nodes
API Endpoint CIDR	TCP ALL	API endpoint to worker node communication
0.0.0.0/0	ICMP 3,4	Path MTU Discovery
Load Balancer CIDR	ALL/10256	LB to kube-proxy health check on worker nodes
0.0.0.0/0 (optional)	ALL/30000-32767	NodePort traffic via LB / Network LB

Worker Nodes subnet — Egress

Destination	Port	Description
Worker Nodes CIDR	ALL	Communication between worker nodes
Pods CIDR	ALL	Worker nodes reach pods on other nodes
0.0.0.0/0	ICMP 3,4	Path MTU Discovery
OCI Services Network	TCP ALL	Nodes communicate with OKE
API Endpoint CIDR	TCP 6443	Worker to API endpoint communication
API Endpoint CIDR	TCP 12250	Worker to API endpoint communication
0.0.0.0/0 (optional)	TCP ALL	General outbound internet access

Pods subnet — Ingress

Source	Port	Description
API Endpoint CIDR	ALL	API endpoint to pod communication
Worker Nodes CIDR	ALL	Pods on one node reach pods on other nodes
Pods CIDR	ALL	Pods communicate with each other — enables the 4 microservices to call each other

Pods subnet — Egress

Destination	Port	Description
Pods CIDR	ALL	Pods communicate with each other
OCI Services Network	ICMP 3,4	Path MTU Discovery
OCI Services Network	TCP ALL	Pods communicate with OCI services
API Endpoint CIDR	TCP 6443	Pod to API endpoint communication
API Endpoint CIDR	TCP 12250	Pod to API endpoint communication
0.0.0.0/0 (optional)	TCP 443	Outbound internet access, if needed

Load Balancer subnet — Ingress

Source	Port	Description
0.0.0.0/0	TCP 443	Public traffic to the Load Balancer

Load Balancer subnet — Egress

Destination	Port	Description
Worker Nodes CIDR	ALL/30000-32767	LB forwards traffic to worker nodes
Worker Nodes CIDR	ALL/10256	LB communicates with kube-proxy health check

optional for troubleshooting: SSH (TCP 22) inbound to worker nodes.

3. Compute: OKE Cluster & Node Pool

Item	Decision
Node shape	TBD, but maybe VM.Standard.A1.Flex (ARM, Always Free eligible)
Node count / OCPU / memory split	TBD
Fixed constraint	Minimum 2 worker nodes regardless of final sizing, so pod replicas have real node-level redundancy

4. Scalability: Replica Strategy (HPA)

Each microservice uses a Horizontal Pod Autoscaler (HPA) with a minimum and maximum replica count rather than a fixed number. This way it keeps the deployed environment small and low-cost for realistic class-project traffic, while demonstrating production-grade scaling design.

Service	Min	Max	Reasoning
Auth	1	3	Every request depends on it, so it needs headroom under load
Product	1	3	Constant read traffic from browsing
Cart	1	2	Frequent but tolerant of brief unavailability
Purchase	2	3	Financial data, never allowed to run on a single replica

5. Traffic Routing: Load Balancer & Ingress

A single OCI Load Balancer sits in the Load Balancer subnet and forwards all traffic to one Ingress Controller. The Ingress Controller performs path-based routing to the correct

microservice, giving the frontend one single API entry point instead of 4 separate public load balancers.

Path	Routed to
/api/auth/*	Auth service
/api/products/*	Product service
/api/cart/*	Cart service
/api/purchase/*	Purchase service

6. Container Registry (OCIR)

Each microservice has its **own OCIR repository**, matching the 1:1 microservice design:

- groupa-auth-app
- groupa-product-app
- groupa-cart-app
- groupa-purchase-app

7. Storage: Database-per-Service

Each microservice owns structured, relational data , so each service uses its own database rather than a shared Block Volume.

- 4 separate databases, one per microservice. no shared database across services
- Isolation Reasoning: Purchase's permanent financial records must stay isolated from Cart's high-churn temporary data
- Each database persists data via its own PVC (Block Volume), following the Week 3 storage pattern
- Specific database engine per service: TBD, left to the implementation team as an implementation detail

8. Security

Area	Design decision
Network isolation	Only the Load Balancer subnet is public-facing; Worker Nodes and Pods subnets are private
Access control	NSGs used for granular, per-resource rules rather than broad security lists
API access	Kubernetes API endpoint access restricted to known team IPs, not 0.0.0.0/0
Secrets	Database credentials and API keys stored as <i>Kubernetes Secrets</i> backed by <i>OCI Vault</i> , never hardcoded in code, images, or Git
IAM	Worker nodes granted a narrowly-scoped dynamic group policy (e.g. OCIR image pulls, Vault reads), not broad compartment-admin rights

9. Automation

This project reuses and extends Week 3's Terraform module structure (subnet module, OKE module, etc.) rather than rebuilding infrastructure from scratch.